

TD-TP : Réseaux de neurones artificiels

Objectifs pédagogiques

À l'issue de ce TD-TP, l'étudiant devra être capable de :

- comprendre le fonctionnement d'un neurone artificiel ;
- implémenter différentes fonctions d'activation ;
- calculer la fonction *Log Loss* ;
- comprendre la convexité de la *Log Loss* ;
- réaliser une propagation avant (*Forward Propagation*) ;
- dériver les gradients par rétropropagation ;
- implémenter l'algorithme de *Backpropagation* ;
- entraîner un réseau de neurones avec la descente de gradient ;
- visualiser la convergence du modèle ;
- évaluer les performances du réseau.

Organisation du TD-TP

- 1 Neurone artificiel
- 2 Fonctions d'activation
- 3 Fonction Log Loss
- 4 Forward Propagation
- 5 Backpropagation
- 6 Entraînement par descente de gradient
- 7 Application au problème XOR
- 8 Évaluation du modèle
- 9 Vérification numérique des gradients
- 10 Application à un jeu de données réel

Environnement

Python + NumPy + Matplotlib + Scikit-learn

Partie I – Le neurone artificiel

Un neurone artificiel calcule :

$$z = w^T x + b$$

puis applique une fonction d'activation :

$$a = f(z).$$

où :

- x : vecteur d'entrée ;
- w : vecteur des poids ;
- b : biais ;
- z : potentiel du neurone ;
- f : fonction d'activation ;
- a : sortie du neurone.

Exercice 1 – Neurone artificiel

On considère :

$$z = w_1x_1 + w_2x_2 + b.$$

Avec :

$$x_1 = 2, \quad x_2 = 3,$$

$$w_1 = 0.5, \quad w_2 = -0.2,$$

$$b = 0.1.$$

Questions :

- 1 Calculer z .
- 2 Calculer la sortie du neurone avec une fonction Sigmoidé.
- 3 Quel est le rôle du biais ?
- 4 Quel est le rôle des poids ?

Partie II – Fonctions d'activation

Les fonctions étudiées sont :

- Sigmoidé ;
- Tanh ;
- ReLU.

Question

Pourquoi une fonction d'activation est-elle nécessaire dans un réseau de neurones multicouche ?

Fonction Sigmoidale

La fonction Sigmoidale est définie par :

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Sa dérivée est :

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Propriétés :

- $\sigma(z) \in (0, 1)$;
- utile pour représenter une probabilité ;
- particulièrement adaptée à la sortie d'une classification binaire ;
- risque de saturation pour des valeurs très grandes ou très petites.

Fonction Tanh

La fonction Tanh est définie par :

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}.$$

Sa dérivée :

$$\tanh'(z) = 1 - \tanh^2(z)$$

et :

$$\tanh(z) \in (-1, 1).$$

Fonction ReLU

La fonction ReLU est définie par :

$$\text{ReLU}(z) = \max(0, z).$$

Sa dérivée est :

$$\text{ReLU}'(z) = \begin{cases} 0, & z < 0, \\ 1, & z > 0. \end{cases}$$

Question

Pourquoi ReLU est-elle très utilisée dans les couches cachées des réseaux de neurones profonds ?

Exercice 2 – Implémentation

Implémenter les fonctions suivantes :

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(z):
    s = sigmoid(z)
    return s * (1 - s)

def tanh(z):
    return np.tanh(z)

def tanh_derivative(z):
    return 1 - np.tanh(z)**2

def relu(z):
    return np.maximum(0, z)
```

Exercice 3 – Visualisation

Utiliser :

```
import matplotlib.pyplot as plt

z = np.linspace(-10, 10, 500)

plt.plot(z, sigmoid(z), label="Sigmoid")
plt.plot(z, tanh(z), label="Tanh")
plt.plot(z, relu(z), label="ReLU")

plt.xlabel("z")
plt.ylabel("f(z)")
plt.legend()
plt.grid()
plt.show()
```

Questions :

- 1 Comparer les trois fonctions.
- 2 Identifier les zones de saturation.

Partie III – Fonction Log Loss

Pour une classification binaire :

$$y \in \{0, 1\}$$

et une probabilité prédite :

$$p \in (0, 1),$$

la fonction Log Loss est :

$$L(y, p) = - [y \log(p) + (1 - y) \log(1 - p)]$$

Interprétation de la Log Loss

Si :

$$y = 1,$$

alors :

$$L = -\log(p).$$

Si :

$$y = 0,$$

alors :

$$L = -\log(1 - p).$$

Interprétation

Une prédiction correcte et très confiante produit une faible perte, alors qu'une prédiction incorrecte et très confiante produit une forte perte.

Exercice 4 – Implémentation de la Log Loss

```
def log_loss(y, p):  
  
    epsilon = 1e-8  
  
    p = np.clip(  
        p,  
        epsilon,  
        1 - epsilon  
    )  
  
    return -(  
        y * np.log(p)  
        +  
        (1-y) * np.log(1-p)  
    )
```

Tester :

```
print(log_loss(1, 0.9))
```

Exercice 5 – Convexité de la Log Loss

On considère :

$$L(p, y) = -[y \log(p) + (1 - y) \log(1 - p)].$$

La dérivée première est :

$$\frac{\partial L}{\partial p} = -\frac{y}{p} + \frac{1 - y}{1 - p}.$$

La dérivée seconde est :

$$\boxed{\frac{\partial^2 L}{\partial p^2} = \frac{y}{p^2} + \frac{1 - y}{(1 - p)^2}}$$

Comme :

$$p \in (0, 1)$$

et :

Partie IV – Forward Propagation

Considérons le réseau :

$$2 \rightarrow 4 \rightarrow 1$$

avec une couche cachée ReLU et une sortie Sigmoidé.

La première couche calcule :

$$Z^{(1)} = XW^{(1)} + b^{(1)}$$

puis :

$$A^{(1)} = \text{ReLU}(Z^{(1)}).$$

La couche de sortie :

$$Z^{(2)} = A^{(1)}W^{(2)} + b^{(2)}$$

et :

$$\hat{Y} = \sigma(Z^{(2)}).$$

Architecture du réseau

$$X \longrightarrow Z^{(1)} \longrightarrow A^{(1)} \longrightarrow Z^{(2)} \longrightarrow \hat{Y}$$

avec :

$$X \rightarrow \underbrace{\text{Dense} + \text{ReLU}}_{4 \text{ neurones}} \rightarrow \underbrace{\text{Dense} + \text{Sigmoid}}_{1 \text{ neurone}}.$$

La prédiction finale est une probabilité :

$$\hat{y} \in (0, 1).$$

Exercice 6 – Problème XOR

Utiliser :

```
x = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
], dtype=float)

y = np.array([
    [0],
    [1],
    [1],
    [0]
], dtype=float)
```

Questions :

- 1 Visualiser les données.
- 2 Pourquoi XOR n'est-il pas linéairement séparable ?

Exercise 7 – Initialisation

```
np.random.seed(42)

W1 = np.random.randn(2, 4) * 0.5
b1 = np.zeros((1, 4))

W2 = np.random.randn(4, 1) * 0.5
b2 = np.zeros((1, 1))
```

Vérifier :

$$W^{(1)} \in \mathbb{R}^{2 \times 4}$$

$$W^{(2)} \in \mathbb{R}^{4 \times 1}.$$

Exercise 8 – Forward Propagation

```
def forward(X, W1, b1, W2, b2):  
  
    Z1 = np.dot(X, W1) + b1  
  
    A1 = relu(Z1)  
  
    Z2 = np.dot(A1, W2) + b2  
  
    A2 = sigmoid(Z2)  
  
    return Z1, A1, Z2, A2
```

Tester :

```
Z1, A1, Z2, A2 = forward(  
    X, W1, b1, W2, b2  
)  
  
print(A2)
```

Partie V – Backpropagation

Pour la sortie Sigmoid + Log Loss :

$$dZ^{(2)} = A^{(2)} - Y$$

Puis :

$$dW^{(2)} = \frac{1}{m} (A^{(1)})^T dZ^{(2)}$$

et :

$$db^{(2)} = \frac{1}{m} \sum dZ^{(2)}$$

Propagation du gradient vers la couche cachée

On calcule :

$$dA^{(1)} = dZ^{(2)}(W^{(2)})^T.$$

Puis :

$$dZ^{(1)} = dA^{(1)} \odot \text{ReLU}'(Z^{(1)}).$$

Enfin :

$$dW^{(1)} = \frac{1}{m} X^T dZ^{(1)}$$

et :

$$db^{(1)} = \frac{1}{m} \sum dZ^{(1)}$$

Exercise 9 – Backpropagation

```
def backward(X, y, Z1, A1, A2, W2):  
  
    m = X.shape[0]  
  
    dZ2 = A2 - y  
  
    dW2 = (1/m) * np.dot(A1.T, dZ2)  
  
    db2 = (1/m) * np.sum(  
        dZ2,  
        axis=0,  
        keepdims=True  
    )  
  
    dA1 = np.dot(dZ2, W2.T)  
  
    dZ1 = dA1 * relu_derivative(Z1)
```

Partie VI – Descente de gradient

Les paramètres sont mis à jour selon :

$$W \leftarrow W - \eta \nabla_W L$$

et :

$$b \leftarrow b - \eta \nabla_b L$$

où η représente le taux d'apprentissage.

Question

Quel est l'effet d'un taux d'apprentissage trop faible ou trop élevé ?

Exercice 10 – Entraînement

```
def binary_cross_entropy(y, p):  
    # Meme formule que log_loss, mais sans clipping  
    return -np.mean(  
        y * np.log(p + 1e-8)  
        + (1 - y) * np.log(1 - p + 1e-8)  
    )  
  
learning_rate = 0.1  
epochs = 10000  
  
loss_history = []  
  
for epoch in range(epochs):  
  
    Z1, A1, Z2, A2 = forward(  
        X, W1, b1, W2, b2  
    )
```

Exercice 11 – Courbe de convergence

```
plt.plot(loss_history)

plt.xlabel("Epoch")
plt.ylabel("Log Loss")

plt.title("Evolution de la fonction de perte")

plt.grid()
plt.show()
```

Questions :

- 1 La fonction de perte diminue-t-elle ?
- 2 Le réseau converge-t-il ?
- 3 Que se passe-t-il si η augmente ?
- 4 Que se passe-t-il si η diminue ?

Partie VII – Prédiction

Après entraînement :

$$\hat{Y} = \text{Forward}(X).$$

La probabilité est transformée en classe :

$$\hat{y} = \begin{cases} 1 & \text{si } \hat{Y} \geq 0.5, \\ 0 & \text{sinon.} \end{cases}$$

En Python :

```
predictions = (A2 >= 0.5).astype(int)
```

Exercice 12 – Accuracy

```
accuracy = np.mean(predictions == y)

print(f"Accuracy = {accuracy * 100:.2f}%")
```

Question

Pourquoi l'accuracy peut-elle être trompeuse lorsqu'un jeu de données est fortement déséquilibré ?

Partie VIII – Comparaison des activations

Réaliser trois expériences :

Activation	Loss finale	Accuracy	Epochs
ReLU			
Tanh			
Sigmoïde			

Questions :

- 1 Quelle activation converge le plus rapidement ?
- 2 Quelle activation donne les meilleures performances ?
- 3 Pourquoi ReLU est-elle généralement utilisée dans les couches cachées ?
- 4 Pourquoi Sigmoïde est-elle utilisée en sortie ?

Partie IX – Vérification numérique du gradient

La dérivée peut être vérifiée numériquement par différences finies :

$$\frac{\partial L}{\partial w} \approx \frac{L(w + \epsilon) - L(w - \epsilon)}{2\epsilon}$$

avec :

$$\epsilon = 10^{-5}.$$

Travail demandé

Comparer le gradient obtenu par Backpropagation avec le gradient obtenu numériquement.

Erreur relative du gradient

Calculer :

$$Error = \frac{|g_{\text{backprop}} - g_{\text{numrique}}|}{|g_{\text{backprop}}| + |g_{\text{numrique}}| + 10^{-8}}.$$

Question

Pourquoi la vérification numérique des gradients est-elle importante lors de l'implémentation d'un nouvel algorithme d'apprentissage ?

Partie X – Application à un jeu de données réel

Utiliser le jeu de données :

Breast Cancer Wisconsin

disponible dans Scikit-learn.

```
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()

X = data.data
y = data.target.reshape(-1, 1)
```

Le jeu contient plusieurs variables décrivant les observations.

Séparation des données

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Standardiser les variables :

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

Architecture proposée

Construire le réseau :

$$30 \rightarrow 16 \rightarrow 8 \rightarrow 1$$

avec :

$$ReLU \rightarrow ReLU \rightarrow Sigmoid.$$

La sortie représente :

$$P(Y = 1|X).$$

Évaluation du modèle

Calculer :

- Accuracy ;
- Precision ;
- Recall ;
- F1-score ;
- matrice de confusion.

La matrice de confusion permet d'identifier :

TP , TN , FP , FN .

Code d'évaluation

```
from sklearn.metrics import (  
    accuracy_score,  
    precision_score,  
    recall_score,  
    f1_score,  
    confusion_matrix  
)  
  
print(  
    "Accuracy:",  
    accuracy_score(y_test, y_pred)  
)  
  
print(  
    "Precision:",  
    precision_score(y_test, y_pred)  
)
```

Questions de synthèse

- 1 Quel est le rôle d'une fonction d'activation ?
- 2 Quel est le rôle du biais ?
- 3 Pourquoi utilise-t-on ReLU dans les couches cachées ?
- 4 Pourquoi utilise-t-on Sigmoidé dans la sortie binaire ?
- 5 Quel est le rôle de la Log Loss ?
- 6 Pourquoi la Log Loss est-elle convexe par rapport à p ?
- 7 Quelle est la différence entre Forward Propagation et Backpropagation ?
- 8 Quel est le rôle du learning rate ?
- 9 Qu'est-ce que le vanishing gradient ?
- 10 Pourquoi standardiser les variables d'entrée ?

Défi avancé – Régularisation L2

Ajouter une régularisation L2 :

$$L_{\text{total}} = L_{\text{LogLoss}} + \frac{\lambda}{2m} \left(\|W^{(1)}\|_F^2 + \|W^{(2)}\|_F^2 \right)$$

Modifier les gradients :

$$dW^{(l)} \leftarrow dW^{(l)} + \frac{\lambda}{m} W^{(l)}.$$

Tester :

$$\lambda \in \{0, 10^{-4}, 10^{-3}, 10^{-2}\}.$$

Défi avancé – Analyse

Comparer les modèles avec et sans régularisation.

λ	Train Loss	Test Loss	F1-score
0			
10^{-4}			
10^{-3}			
10^{-2}			

Questions :

- La régularisation réduit-elle le surapprentissage ?
- Quelle valeur de λ donne le meilleur compromis ?
- Pourquoi une régularisation trop forte peut-elle dégrader les performances ?

Projet final

Construire un réseau complet :

$X \rightarrow \text{Dense} \rightarrow \text{ReLU} \rightarrow \text{Dense} \rightarrow \text{ReLU} \rightarrow \text{Dense} \rightarrow \text{Sigmoid} \rightarrow \text{LogLoss}$

Le programme doit réaliser :

- 1 chargement des données ;
- 2 prétraitement ;
- 3 initialisation des paramètres ;
- 4 Forward Propagation ;
- 5 calcul de la Log Loss ;
- 6 Backpropagation ;
- 7 calcul des gradients ;
- 8 mise à jour des paramètres ;
- 9 répétition pendant plusieurs epochs ;
- 10 évaluation ;
- 11 visualisation de la convergence.

Livraison du TD-TP

Le compte rendu doit contenir :

- 1 le code Python complet ;
- 2 les fonctions d'activation et leurs dérivées ;
- 3 l'implémentation de la Log Loss ;
- 4 les équations du Forward Propagation ;
- 5 la dérivation du Backpropagation ;
- 6 la courbe Loss-Epochs ;
- 7 les performances de classification ;
- 8 la matrice de confusion ;
- 9 la comparaison ReLU/Tanh/Sigmoïde ;
- 10 la vérification numérique des gradients ;
- 11 une analyse des résultats.

Conclusion

Ce TD-TP permet de relier les concepts mathématiques et leur implémentation informatique :

Fonction d'activation → Forward Propagation → Log Loss → Backpropagation → Descente de gradient

L'étudiant dispose ainsi des bases nécessaires pour comprendre l'entraînement des réseaux de neurones modernes.

Objectif final

Être capable d'implémenter et d'expliquer mathématiquement un réseau de neurones sans dépendre d'une bibliothèque de Deep Learning.

Fin du TD-TP

Questions et discussion